

그룹배관패턴 자동경로 — 단계별 프로세스·알고리즘 상세

마지막 갱신: 2026-06-06 · 단위 mm · 대상: `csharp/Routing3D.Viewer`

관련 소스: `SceneViewModel.cs` · `BundleStore.cs` · `DbRouteDiag.cs`

학습 파이프라인: `python_experiments/routing3d_py/bundle_detect.py` · `db/schema/route_bundle_group.sql`

0. 한눈에 보기

"그룹배관패턴 경유"는 사람이 설계한 기존배관(`TB_ROUTE_PATH`)의 '번들(평행 다발)' 형태를 학습해, 같은 유틸리티의 신규 배관이 그 번들 골격(공용 트렁크 + 수직 입상 + 꺾임 코너)을 따라가도록 만드는 경로 방식이다.

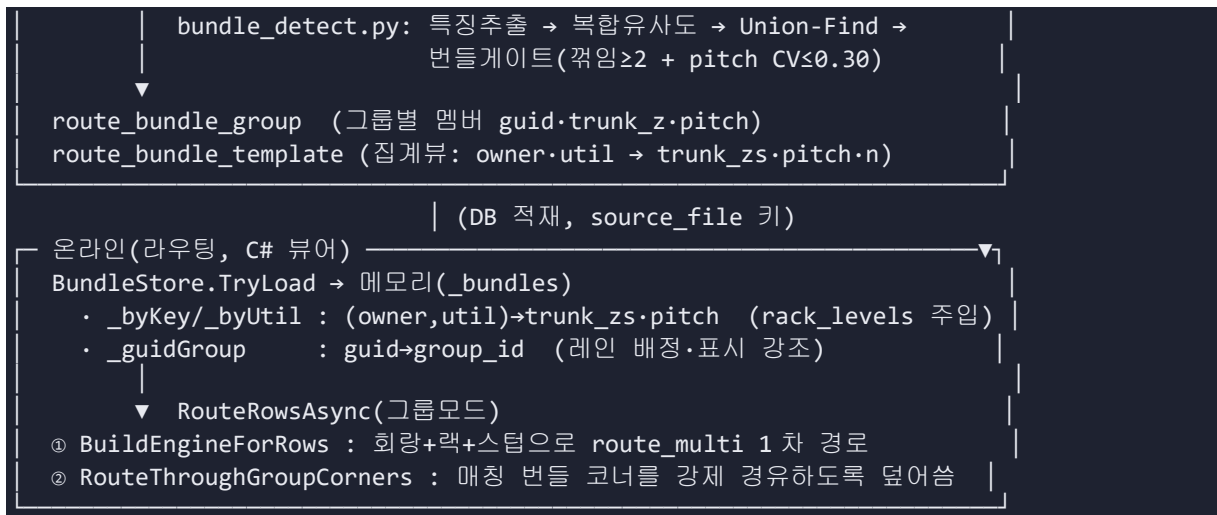
3 개의 경로 방식 중 하나다:

방식	RouteMode	후처리 메서드	성격
최단경로	<code>Shortest</code>	없음	순수 A* (회랑/패턴 모두 OFF)
그룹패턴 경유	<code>GroupPattern</code>	<code>RouteThroughGroupCorners</code>	번들 코너만 강제 경유 + 사이는 자유
기존설계 추종	<code>FollowExisting</code>	<code>ReplicateMatchedPipes</code>	매칭 배관 폴리라인 전체 복제

핵심 원칙은 "코너는 강제·사이는 자유": 기존배관의 꺾임점(코너)만 반드시 통과하고, 코너 사이는 충돌 없는 최단 직교로 잇는다. 그래서 그룹 골격은 따르되 불필요한 우회는 없다.

1. 두 시점 — 오프라인 학습 vs. 온라인 라우팅

오프라인(학습, Python) —
`TB_ROUTE_PATH` (사람 설계배관 폴리라인)



라우팅은 2 단계다. ① 번들 회랑/랙으로 부드럽게 유도한 `route_multi` 1 차 경로를 만든 뒤, ② 매칭되는 번들 멤버의 코너 **waypoint** 를 강제 경유하는 경로로 덮어쓴다. ①은 충돌회피(mark_pipe)·다발 분산을 담당하고, ②는 패턴 충실도를 담당한다.

2. 오프라인 학습 — 번들 탐지 (`bundle_detect.py`)

신규 라우팅이 따라갈 "번들"을 먼저 DB 에 만들어 둔다. 3 단계:

2.1 특징 추출

각 기존배관(폴리라인)에서:

- **방향 런 압축**: 연속 동일축 구간을 하나의 런으로 (지터 흡수)
- **Arrow 시퀀스**: 각 런을 R(수평행)/H(수평횡)/D(수직)로 부호화
- **꺾임 수, 리샘플 방향벡터**(고정 N 개), **extent**(bbox 규모)

2.2 복합 유사도

두 배관의 유사도 = 가중합:

- 형태 30% (Arrow 시퀀스 Levenshtein)
- 방향 30% (리샘플 방향벡터 코사인)
- 길이 20%
- 규모 20% (extent 비율)

2.3 그룹화 + 번들 게이트

1. `(owner_name, utility)` 로 **pre-filter** (같은 장비·유틸끼리만 비교)
2. **Union-Find** (유사도 임계 0.70 이상 병합)
3. **번들 게이트**: 그룹이 "번들"로 인정되려면

- 멤버 간 꺾임 ≥ 2 회 공유
- 이격간격(pitch) 변동계수 $CV \leq 0.30$ (등간격 평행 다발)

1. 통과 그룹 → **trunk_z**(공용 수평선 고도), **다발폭**, **pitch** 산출

2.4 산출물

테이블	내용
<code>route_bundle_group</code>	그룹별 <code>group_id</code> , <code>member_guids</code> (text[]), <code>trunk_axis</code> (0/1/2), <code>trunk_zs</code> , <code>pitch_mm</code>
<code>route_bundle_template</code> (집계뷰)	<code>(owner_name, utility) → trunk_zs, pitch_mm, n_members</code>

실측: project6 전체 70 프로젝트·353 그룹·275 키 적재. project1(CMP_KSCTA08) = 22 그룹·121 멤버. ALKA 유틸은 수직 번들(`trunk_axis=2`, `gid4=8` 멤버·`gid6=6` 멤버, `pitch` 180mm).

CLI:

```
python -m routing3d_py.bundle_detect --project 1 --write-db # 단일 프로젝트 적재
python -m routing3d_py.bundle_detect --all --write-db # DB 전체 순회
python -m routing3d_py.bundle_detect --templates # 집계뷰 확인
```

3. 온라인 — 번들 저장소 로드 (`BundleStore.TryLoad`)

뷰어가 프로젝트를 로드할 때 `source_file` 키로 `_bundles` 를 메모리에 올린다.
(`BundleStore.cs:52`)

세 개의 인덱스를 만든다:

- `_byKey[(owner,util)]` → 대표 트렁크 고도·pitch (`rack_levels` 주입용)
- `_byUtil[util]` → 유틸 단위 폴백(트렁크 고도 합집합)
- `_guidGroup[guid]` → `group_id` (레인 배정·표시 강조용), `_groupCount`

⚠ MARS 주의 (과거 버그)

Npgsql 은 MARS(한 커넥션에 동시 reader 다중)를 지원하지 않는다. 첫 `reader(route_bundle_template)`를 `using var` 로 열어두면, 같은 커넥션에서 두 번째 `reader(route_bundle_group)`를 열 때 예외 → 그룹 멤버십(`_guidGroup/_groupCount`)이 통째로 유실된다(레인 배정·표시 무력화, `lane 0/0 gc0`). 그래서 첫 reader 는 반드시 블록 스코프 `using(...){ }` 로 닫고 두 번째를 연다. ([BundleStore.cs:69-93](#))

테이블 부재/연결 불가 → `null` 반환 → 뷰어는 번들 미적용으로 무해 폴백(회귀 0).

4. 1 단계 — 번들 유도 1 차 경로 (`BuildEngineForRows`, `groupMode=true`)

[SceneViewModel.cs:1084](#). 엔진을 [장애물 전체 + 지정 행 작업]으로 재구성한다. 그룹 모드의 핵심 4 가지:

4.1 weighted A* + 동적 가중

- 격자 > 300k 셀이면 weighted A* (`w_heur=2.0`) — 솔리드 설비/덕트가 많은 혼잡 격자에서 목표지향 탐색.
- 동적(수렴) 가중(`w_heur_near=1.0`): 휴리스틱 가중을 목표까지 거리비로 보간 — 먼 곳은 2.0(빠름), 목표 근처는 1.0(표준 A*, 정확). 목표/PoC 근처 혼잡·막다른길의 그리디 함정 회피.

4.2 랙 레벨 주입 (`rack_levels =` 면제 z-셀)

- `BuildRackLevels` (L3a, 유틸그룹 수평런 학습 z) +
- `MergeBundleLevels` (L4): `_bundles.TryGet(null, util).TrunkZs` → z-셀로 변환해 합침. 같은 유틸 배관이 학습된 공용 랙 높이에 뭉친다. ([SceneViewModel.cs:1436](#))
- 랙 페널티 = `cell × 0.2` (회랑보다 부드러움).

4.3 번들 트렁크 회랑 (`BuildBundleCorridorCells`, `includeVertical=true`)

- 탐지된 번들 멤버 배관의 모든 런(수평+수직 입상)을 타이트(± 1 셀) 레인 셀로 만들어 회랑 시드로 주입. ([SceneViewModel.cs:1252](#))
- 표시([ShowBundlePattern](#))와 라우팅 회랑이 같은 셀집합 공유 → 보이는 보라색 패턴이 곧 신규경로 경유지.
- 회랑 패널티 = cell $\times$ 0.5 (랙보다 강한 설계추종). `w_corridor > 0` + 공유 회랑이면 route_multi 의 self-bundling(mark_pipe + add_corridor_cells)이 둘째 배관을 첫 배관 곁으로 뭉친다(test_attract 가 증명).
- 주의: 셀 > pitch 면 인접 레인이 같은 셀로 뭉개진다(cell=100 > pitch $\approx$ 56) → 셀 $\leq$ pitch/2 권장(25~50mm).

4.4 스텝 라우팅 (per-task)

- 매칭 기존배관의 출발/종단 스텝(수직+엘보)을 고정 설계 구간으로 깔고, A*는 스텝 끝(랙 위 자유공간)~끝만 탐색. ([SceneViewModel.cs:1160](#))
- 폴백(매칭 없음): `DropStartBelowEquipment` → `LiftPocToSurface`(학습면) → `SnapPocToFreeCell`(파묻힌 PoC 구제).

이 엔진으로 `RouteMulti("utility")`를 실행 — 같은 유틸을 묶어 순차 라우팅하면 self-bundling 이 유틸별로 일관되게 자란다. ([RouteRowsAsync](#), [SceneViewModel.cs:2305](#))

셀 자동 축소: 그룹 라우팅 '다건'(2 개 이상)은 셀 $\leq 50\text{mm}$ 라야 인접 레인이 분리된다. DB 프로젝트 로드 상태에서 `_cellMm > 50` 이면 50mm 로 재적재한다. ([SceneViewModel.cs:2251](#))

5. 2 단계 — 코너 강제 경유 ([RouteThroughGroupCorners](#))

[SceneViewModel.cs:1878](#). 1 차 경로를 번들 코너 경유 경로로 덮어쓴다. 매칭 없는 행은 1 차 A* 결과를 유지한다.

5.0 격자 가드

```
if ((long)g.Nx * g.Ny * g.Nz > 300_000_000L) return -1; // 생략(A* 유지)
```

과거 30M 가드 + `r3d_route_task` 가 Dense 전용이라 cell=25(1.3 억 셀)에서 패턴이 통째로 생략되던 버그가 있었다. → 엔진 `r3d_route_task` 를 `route_multi` 처럼 5M 초과 시

ImplicitOccupancy 자동 전환으로 고치고(PR #21), 가드를 300M 으로 올려 기본 cell=25 가 동작하게 했다.

5.1 수리 엔진 + 막힘 판정 (Blocked)

- `rep` Engine = 장애물 + 설비만(배관 마크 없음) → 코너 사이 국소 A* 재사용. 마크가 없으니 같은 그룹 배관이 자기 레인 코너를 따라가며 자연 다발.
- `Blocked(ci,cj,ck)` = 장애물(통과 제외)+설비+덕트 박스를 $\min T(=cell)$ 로 팽창한 AABB 와 셀이 겹치면 막힘.

5.2 번들 레인 배정 (AssignBundleLanes)

`SceneViewModel.cs:1946`. 같은 번들 그룹의 작업들을 서로 다른 멤버 배관에 1:1 배정 → 공유 트렁크를 평행 다발로 재현(레인 붕괴 방지).

알고리즘:

1. 각 작업 → 최근접 매칭 배관의 `group_id` (`_bundles.GroupIdOf`)
2. 그룹별 멤버 배관 목록 수집
3. 2 개 이상 작업 그룹에 대해 탐욕 1:1:
 - 모든 (작업, 멤버) 쌍의 비용 = $\min(d(ts,ps)+d(te,pe), d(ts,pe)+d(te,ps))$ (양방향 정렬)
 - 비용 오름차순 정렬 → 둘 다 미사용인 전역 최소쌍부터 배정
 - 멤버보다 작업이 많으면 남은 작업은 최근접 멤버 재사용(회랑/마크가 레인 분리)
1. 단일 작업 그룹.번들 미적재.미소속 → 배정 안 함(호출자가 최근접 매칭으로 폴백, 회귀 0)

5.3 코너 waypoint 추출 (GroupCornerWaypoints)

`SceneViewModel.cs:2013`. 배정된 멤버 배관에서 축 전환 코너만 뽑는다:

1. 작업 PoC 방향에 맞춰 폴리라인 정렬(`fwd` 판정, 필요시 reverse)
2. 각 정점에서 `Axis(prev,cur) != Axis(cur,next)`면 코너
3. 짧은 런(<250mm)의 챔퍼/지터 코너는 인접 코너로 흡수(주요 코너만)
4. 결과 = [작업 start ts] + 코너들 + [작업 end te] (양 끝은 작업 실제 PoC)

5.4 코너 폴리라인 → 셀 경로 (ReplicateCellPath)

`SceneViewModel.cs:2062`. waypoint 정점쌍을 차례로 잇는다:

정점들을 셀로 변환(중복 제거)

```

선두 막힘(물힌 시작 PoC) 스킵
각 (prev → target) 쌍:
    target 막힘(물힌 코너) 스킵
    L = FreeOrthoL(prev, target)          ← 충돌 없는 직교 L (양 측순서 시도)
    if L != null: outp += L
    else:                                ← 둘 다 막힘
        repair = RepairAStar(prev, target)
        repair = StraightenOrtho(repair) ← 수리 톱니만 정리
        outp += repair
return DeJog(outp)                        ← 말림(짧은 jog)만 제거

```

코너(정점)는 앵커로 보존되므로 그룹패턴 코너를 강제 경유한다.

6. 핵심 직교 연결 알고리즘 3 종

이 세 함수가 "패턴 충실도 ↔ 깔끔한 밴딩" 균형의 핵심이다.

6.1 FreeOrthoL — 충돌 없는 직교 L (양 측순서)

SceneViewModel.cs:2173. A→B 가

- 1 축(직선): 그대로 채움
- 2 축(L): 두 축 순서를 모두 시도, 충돌 없는 첫 L 반환
- 0 축/3 축: null

핵심: 한 측순서만 쓰면(긴 축 우선) 그 L 이 막혔을 때 불필요한 A* 톱니가 생긴다. 반대 순서 L 이 자유면 그걸 써 2 런으로 깔끔히 잇는다. (과거 "꾸불꾸불 계단" 회귀의 근본 해결)

6.2 DeJog — 말림(짧은 jog)만 제거 ★현재 후처리

SceneViewModel.cs:2104. 같은 축·방향 두 런 사이에 긴 ≤ 4 셀(cell=25→100mm) 다른축 런 = 계단 한 칸(말림)만 충돌 없는 단일 L 로 흡수 → "한 번에 꺾이는" 엘보.

```

반복(최대 128 회):
    경로를 런으로 분해 (축, 방향, 시작 idx, 끝 idx)
    for 각 중간 런 i:
        if runs[i-1].축==runs[i+1].축 && 방향 같음 && runs[i].길이 ≤ 4:
            L = FreeOrthoL(런 i-1 시작, 런 i+1 끝)
            if L: 그 구간을 L 로 교체; 다시 반복
    변화 없으면 종료

```

왜 DeJog 인가 (과거 회귀 교훈): 처음엔 전체 경로에 StraightenOrtho(임의 2 축 자유 L 단축)를 적용했으나, 수평↔수평 코너도 2 축이라 개방공간에서 단축되어 → 설계 코너 절반이

붕괴(avgTurns 8.1→4.3) → 패턴이 최단처럼 보임. DeJog 은 긴 런 사이의 큰 패턴 코너는 건드리지 않고 짧은 jog 만 병합하므로 패턴을 보존한다.

6.3 StraightenOrtho — 수리 구간 전용

SceneViewModel.cs:2150. 가장 먼 자유 L 부터 채택해 톱니를 길게 편다. 임의 2 축 L 을 단축하므로 큰 코너도 붕괴 가능 → 막힌 구간 국소 A* 수리 결과(킨크)에만 사용하고, 조립된 전체 경로에는 절대 쓰지 않는다(대신 DeJog).

7. 전체 흐름 (의사코드)

```
RouteRowsAsync(rows, corridor=true, mode=GroupPattern):
    if 다건 && cell>50: cell=50 재적재
    ClearRouteResults(rows); BuildModel()

    — 1 단계 —
    added = BuildEngineForRows(rows, groupMode=true)
        • weighted A* + 동적가중(w_heur 2.0/near 1.0)
        • rack_levels = BuildRackLevels + MergeBundleLevels(L4 trunk_z)
        • corridor = BuildDesignCorridorCells(L2b) ∪ BuildBundleCorridorCells(L4)
        • w_corridor = %cell
        • per-task: 스텝 라우팅(스텝끝~끝) 또는 학습면 폴백
    engine.RouteMulti("utility") ← self-bundling 1 차 경로
    CacheResults(added)

    — 2 단계 —
    RouteThroughGroupCorners(added):
        if 격자>300M: return -1 (생략)
        rep = 수리엔진(장애물+설비, 마크없음); Blocked = 팽창 AABB
        lane = AssignBundleLanes(added) ← 번들 작업↔멤버 1:1
        for 각 작업:
            pipe = lane[작업] ?? FindMatchingExistingPipe(작업)
            wps = GroupCornerWaypoints(작업, pipe) ← 코너만
            path = ReplicateCellPath(wps, Blocked, rep)
                    FreeOrthoL(양축) → 막히면 RepairAStar+StraightenOrtho
                    최종 DeJog(말림 제거)
            작업.Path = path (덮어쓰기)

    BuildModel() ← 최종 렌더
    Status = "성공 N/M · 그룹패턴 경유 K"
```

8. 폴백 사다리 (무해 설계)

각 단계가 실패해도 회귀 0 으로 자연 폴백한다:

실패 지점	폴백
번들 저장소 미적재 (<code>_bundles==null</code> /GroupCount=0)	레인 배정 빈 맵 → 최근접 매칭 → 기하 회랑만
작업이 번들 미소속	최근접 매칭 배관으로 코너 추출
매칭 배관 없음	1 차 A*(최단) 결과 유지
코너 사이 양 축순서 L 모두 막힘	RepairAStar 국소 우회
RepairAStar 실패	<code>ReplicateCellPath</code> null → 1 차 A* 유지
격자 > 300M 셀	그룹패턴 경유 생략(-1), 상태바 안내

9. 진단·검증 (헤드리스)

```
# 그룹패턴 경유 A/B (OUT 파일에 지표 기록 - WPF 는 콘솔 미출력)
Routing3D.Viewer.exe --dbroute 1 25 ALL out.txt
# env: R3D_GROUPCORNER=on R3D_BUNDLE=on R3D_STUB=on 등으로 단계 토글
```

지표 (`[groupcorner matched M ok N avgLen L avgTurns T avgCorners C avgReps R avgJogs J lane D/T]`):

- **avgTurns** $\approx$ **avgCorners** → 패턴 충실(코너 보존 정상). avgTurns $\ll$ corners 면 패턴 붕괴.
- **avgJogs** → 말림 잔여(0 에 가까울수록 깔끔). DeJog 가 줄임.
- **lane D/T** → 번들 레인 분산(D=distinct 멤버, T=총 작업).

실측 (project6, cell=25): ALKA avgTurns 7.7 $\approx$ corners 7.8(패턴 복원)·avgJogs 0.4 / ALL avgTurns 8.3 $\approx$ 7.9·108/110 성공·lane 102/102.

10. UI 토글 정리

토글	효과
경로방식 = 그룹패턴 경우	<code>RouteThroughGroupCorners</code> 활성(<code>_routeWaypoints</code>)
그룹배관 패턴(BundlePattern, 기본 OFF)	<code>rack_levels(trunk_z)</code> + 번들 트렁크 회랑 주입
그룹배관 강조 표시(ShowBundleGroups)	번들 멤버를 그룹별 고유색(황금비), 비멤버 흐리게
진행 다이얼로그 그룹배관패턴(보라 #BE78EB)	선택 배관의 학습 트렁크 라인 미니 3D 표시

미적재/키 미스 시 기존 기하 규칙으로 자동 풀백(무해).

부록 — 왜 2 단계인가? (1 차 회랑 vs. 2 차 코너의 역할 분담)

- **1 차(route_multi + 회랑/랙):** *충돌회피*와 *다발 분산*에 강하다. `mark_pipe` 로 배관 간 충돌 0 을 보장하고, self-bundling 으로 공용 트렁크에 모은다. 그러나 회랑은 "소프트 바이어스"라 코너를 **정확히** 통과한다는 보장은 없다(비용만 낮춤).
- **2 차(코너 강제 경우):** 매칭 번들의 *꺾임 골격을 정확히* 재현한다. 코너를 앵커로 강제하되 사이는 자유 직교라 불필요한 우회가 없다.

둘을 합쳐 "**충돌 없는 다발(1 차) + 정확한 번들 코너(2 차)**"를 동시에 얻는다. 1 차만으로는 코너가 흐려지고, 2 차만으로는 충돌회피·다발 분산이 약하다.